

Alaiy Computer Scientist Skill Test

Automated Garment Catalog Builder (End-to-End System)

Objective

Build a **full-stack system** that converts raw garment photos into a **ready PowerPoint catalog**.

This is **not a script**.

This is a **live system** with:

- A frontend interface
 - A backend processing pipeline
 - Real-time visibility into how data flows and transforms
-

Problem Context

You are given a folder of **unstructured garment images**:

- Front views
- Back views
- Detail shots
- Spec/label cards

These images are:

- Messy
- Unlabelled
- Inconsistent

Your job is to build a system that:

1. Understands them
 2. Organises them
 3. Cleans them
 4. Extracts data
 5. Generates a final catalog
-

Core Requirement (Important)

This **must NOT** be a single API call pipeline.

We want to see:

- Step-by-step processing
- State transitions
- Observability

Implement:

- Webhooks / event-driven architecture / job queues
- OR any equivalent system that shows:
 - Image ingestion
 - Classification
 - Rearrangement
 - Cleanup
 - Final output generation

We should be able to **see every stage of transformation**.

Frontend Requirement (Mandatory)

Build a frontend interface where:

1. Upload System

- Upload multiple images (drag-drop or file picker)
- It should accept 50 styles (3-4 images per style)

2. Visual Workspace (Important)

- Display all uploaded images in a **canvas-like view**
- Similar to a whiteboard (think free layout)
- Images should:
 - Rearrange automatically as classification improves
 - Group into garment styles
 - Update in real-time

3. Live Pipeline View

- Show status of each image:
 - Uploaded
 - Classified

- Processed
 - Cleaned
 - Assigned to slide
 - We should be able to visually track:
 - Movement
 - Grouping
 - Re-grouping
-

Workflow to Implement

Step 1 — Image Classification

- Group images by garment style
 - Identify:
 - Front
 - Back
 - Detail
 - Spec label
-

Step 2 — Spec Data Extraction

From label images, extract:

- Reference Number
- Fabric Composition
- GSM
- Date

Store as structured JSON.

Step 3 — Image Processing

Apply:

- Smart cropping
- Background cleanup (make it visually clean, not destructive)
- Brightness/contrast correction
- Maintain:
 - Original orientation
 - Aspect ratio (NO stretching)

Standardise:

- Front/Back → portrait format
-

Step 4 — PowerPoint Generation

You will be given a **reference PPT**.

Your output must:

- Match layout exactly
- Maintain consistent spacing and alignment

Each slide must contain:

- Front image (left)
 - Back image (center)
 - Detail image (top right)
 - Extracted specs (text)
 - Logo (bottom right)
 - Style name + number (header)
-

Step 5 — File Structuring

Generate a clean output folder:

```
Processed_Garments/  
  StyleName_front.jpg  
  StyleName_back.jpg  
  StyleName_detail.jpg
```

Tech Stack

Backend

- You may use **any stack**

AI Models

- You will be provided with:
 - OpenRouter API key (\$5 credits)
- You may use:
 - Any model available via OpenRouter
 - Any additional tools (clearly document usage)

Important Constraints

- No image stretching
 - Maintain original orientation
 - Pipeline must be observable (no black-box processing)
 - System must handle messy real-world inputs
 - Background cleanup should improve visual quality without damaging garment details
-

Deliverables

1. GitHub Repository (Public)

Must include:

- Clean commit history (no last-day dumps)
 - Backend code
 - Frontend code
-

2. Output Files

- Final PowerPoint ([Catalog.pptx](#))
 - Processed image folder
-

3. Technical Documentation (Minimum 3 Pages)

Include:

System Design

- Architecture
- Data flow
- Event handling (webhooks / queues / pipelines)

Feature Breakdown

- Classification
- Extraction
- Image processing
- Frontend behavior

API / Pipeline Design

- Endpoints / events
- Data structures

Design Diagram

- End-to-end system

Screenshots

- UI
 - Pipeline stages
 - Final output
-

4. Demo Video (1–2 mins)

Show:

- Upload → processing → final output
 - Frontend interactions
 - Pipeline visibility
-

Submission

Send an email to:
pradyun@alaiy.com

CC:

- jayasurya@alaiy.com
- amit@alaiy.com
- sarthak@alaiy.com

Include:

- Short introduction
 - GitHub link
 - Documentation link
 - Demo video link
-

Confidentiality

All data, assets, and problem statements shared are **confidential**.
Do not reuse or publish them outside this assignment.

Evaluation Criteria

1. System Design

- Pipeline clarity
- Observability
- Handling async workflows

2. Technical Quality

- Clean, modular backend
- Scalable design

3. Frontend Experience

- Intuitive
- Real-time updates
- Visual clarity

4. AI Usage

- Smart usage of models
- Not over-reliant on a single call

5. Output Quality

- Professional PPT
- Clean images

6. Documentation

- Depth and clarity
-

Bonus Points

- Real-time updates via websockets
 - Drag-and-drop manual correction in UI
 - Confidence scores for classification
 - Retry/failure handling
 - Batch processing
 - Versioning of outputs
 - Slide preview before export
-

Final Note

You are encouraged to use AI tools (GPT, Claude, Cursor, etc.).
Clearly document:

- Where they were used
- What you built yourself

Even if incomplete, submit your work.

We value **effort, clarity, and problem-solving approach.**
